AI Infra 学习讲义

10.12-10.15 合并版

Serving 可观测性、过载保护、模型热更新

TensorRT 精度、动态 Shape 与 C++ 执行边界

> Mac 阅读版 · A4 纵向 · 以 10 月追赶计划和当前学习分身为基础。四天内容不是把名词拼在一起，而是沿着同一条请求生命线，把测量、诊断、保护、切换和后端执行连起来。

日期范围：2026-10-12 至 2026-10-15 建议总投入：约 30 小时（每天约 7.5 小时）

当前证据边界：仓库已保存 10.11 synthetic sweep 的 2600 行 CSV，覆盖 12 组配置且样本状态均为 ok；它来自 FakeRuntime，不是 ORT/CUDA 容量。当前 CSV 不含每组 elapsed/runtime_calls/batch-size 分布，不能独立重算 QPS 或 batching efficiency。ORT CUDA 也没有新的 WSL Session/parity/节点归属结果。

怎么使用这份合并讲义

这四天的主题可以看作给一个已经能接收请求、排队、合批并调用 Runtime 的服务，加上四类能力：看得见（指标、日志、Trace）、扛得住（限流与过载保护）、换得稳（模型版本与热更新）、跑得快且正确（TensorRT Engine / Context / Stream）。

日期	Serving 主线	TensorRT 追赶槽	当天可交付证据
10.12	Metrics：定义指标、单位、标签与延迟边界	Engine 生命周期与 FP32 基线	Metrics 字典 + FP32 build manifest
10.13	Structured Logging + Distributed Trace	FP16 / TF32 / INT8 精度边界	请求 Trace + 精度 parity 报告
10.14	Admission、过载保护、G7 集成验收	Dynamic Shape 与 min/opt/max Profile	故障注入测试 + Profile 边界矩阵
10.15	模型版本、热切换、Canary 与回滚	C++ Buffer / CUDA Stream / enqueue	版本切换测试 + Runner 时序图

四天贯穿的请求时间线

```
client -> gateway -> admission -> queue -> batcher -> RuntimeAdapter
       -> TensorRT/ORT Session -> result mapping -> serialization -> client
          |          |       |       |        |                |
         trace id   reject  queue  batch wait  compute       response
```

同一个 request_id / trace_id 应能串联日志和 Trace；Metrics 则从大量请求中抽取可聚合的数量与分布。TensorRT 的工作在 RuntimeAdapter 之后，它不负责队列、公平性或网络超时；Serving 也不应把 GPU kernel 的时间冒充端到端 latency。

已知起点与本讲纠偏

- Queue Time 在当前 Inferflow 计时定义中是 enqueued → dequeued。请求 dequeue 后若 parked 等待 deferred batch，它不再属于 Queue Time；该段由 pre_bucket_wait 表达，并计入 E2E。
- Concurrency 是某一时刻的在途请求数，不是每秒到达率，也不必等于 Queue 队列长度。Open-loop 固定到达时刻；closed-loop 固定并发客户端/在途目标。
- 小样本 p99 不能被解释成稳定尾延迟。保留 raw samples、样本量、warmup、重复轮次、percentile method 和误差/置信信息。
- device_resident 只表示输入输出留在设备侧的接口边界；仍会包含 Runtime 调度、Kernel launch、可能的同步与内存分配，不等于纯 kernel。
- 10.11 synthetic CSV 已存在：2600 行、12 组、状态全 ok；它只表示 FakeRuntime 的 closed-loop 样本，不能外推 ORT/CUDA。CSV 缺少 elapsed、runtime_calls 与真实 batch-size，因此吞吐/批处理效率需要额外证据。

学习成果

- 能从一条请求的 timestamp 解释 Queue、Batch Wait、Runtime、Serialization、E2E 的范围，并说出哪些值尚未采集。
- 能区分 Counter、Gauge、Histogram，识别高基数标签，并把慢请求从日志追到跨线程/跨服务的 Trace。
- 能用容量上限、并发限制、超时、拒绝与熔断形成过载保护闭环，并在测试中验证内存有界、future 终态唯一。
- 能说明模型热更新为何要以不可变版本对象/引用计数交换，怎样让旧请求安全用完旧模型并支持回滚。
- 能描述 TensorRT Engine 与 ExecutionContext 的职责、FP32/FP16 等精度比较、Optimization Profile 的 shape 契约和 enqueue/stream/buffer 生命周期。

第 0 章：先把 Serving 指标的时间边界钉牢

Metrics 不会自动让数字正确。先对一条请求定义起点和终点，再把每个间隔命名；否则同名的 queue_time 可能有人算到 dequeue，有人算到 batch dispatch，最后看板无法比较。

```
submitted -> enqueued -> dequeued -> bucketed -> dispatched
                                                    |
                                          runtime_begin -> runtime_end
                                                    |
                              result_ready -> serialized -> completed

queue_time       = dequeued - enqueued
pre_bucket_wait  = bucketed - dequeued   # deferred parking interval
batch_wait       = dispatched - bucketed
runtime_time     = runtime_end - runtime_begin  # IRuntime::infer() wall interval
serialization    = serialized - result_ready   # if not measured, report null/unavailable
end_to_end       = completed - submitted
```

怎样判断这些间隔有没有重叠？

把 timestamp 看成事件而不是独立计时器。若同一请求的阶段严格串行，总 E2E 大致是各阶段时长相加；若 H2D、Compute 或多个请求并行，则不能简单相加。一个 Runtime 的 infer wall interval 可能包含 CPU enqueue、GPU 排队、GPU 执行以及等待策略。不要把 host wall clock、CUDA Event 和 Profiler duration 混成一个单位相同就相同的量。

特别检查 deferred batch

请求入队后被 worker dequeue，发现当前 batch bucket 暂时不能执行，于是 parked 到 deferred list，再在后续时刻并入 batch。若 `dequeued` 已记录，Queue Time 在 dequeue 处结束；从 dequeue 到真正进入 bucket 的这段应明确记为 pre_bucket_wait，避免重复或漏算。需要靠人工读代码确认状态转换点，不要只凭变量名称猜。

> 小练习：写一个请求时间线，令 enqueued=2ms、dequeued=5ms、bucketed=8ms、dispatched=13ms、Runtime=4ms、completed=22ms。于是 Queue=3ms、pre-bucket=3ms、Batch Wait=5ms、Runtime=4ms；若 Serialization 未计时，不能用 E2E 减其他区间伪造 serialization，除非所有阶段互斥、完整且定义一致。

指标命名与失败路径

名称要编码语义，不只写 duration。建议分别保留 `inferflow_queue_wait_seconds`、`inferflow_pre_bucket_wait_seconds`、`inferflow_batch_wait_seconds`、`inferflow_runtime_infer_seconds`、`inferflow_request_e2e_seconds`。若一个区间未测，导出状态/字段应明确 unavailable，而不是写 0；0 表示真实测得零耗时，语义不同。

验证成功、拒绝、过期、取消、Runtime 异常、序列化失败、shutdown drain 几条路径。异常路径也必须给请求唯一终态和完成时间；否则服务可能显示成功请求的延迟很漂亮，实际错误/悬挂请求却被统计漏掉。

10.12：Metrics + TensorRT Engine / FP32

Part A：指标是服务的仪表盘，不是日志的替代品

想象监控一条高速公路：Counter 是累计通过的车辆数；Gauge 是此刻路上车辆数；Histogram 是每辆车用时的分布桶。Counter 适合算某一段时间的增量速率，Gauge 适合看当前队列深度/显存，Histogram 适合看请求延迟和 batch size 分布。

类型	生活类比	Serving 例子	常见误用
Counter	里程表只增不减	requests_total、errors_total、runtime_calls_total	把当前并发当 Counter；进程重启后差值处理错误
Gauge	此刻的油量/人数	queue_depth、inflight_requests、active_model_version_info	把累计请求数当 Gauge；采样太慢漏掉瞬时峰值
Histogram	按时长区间统计车辆	request_e2e_seconds、batch_rows、queue_wait_seconds	只存平均数；桶太粗或边界不足，看不到尾延迟

为什么平均值不能替代 p95？

若 100 个请求中 95 个花 10ms、5 个花 1 秒，平均值约 59.5ms。只看平均，服务看起来还算快，但每 20 个请求就有 1 个非常慢。Histogram 保存分桶计数，才能估算分位数并比较尾部；分位数的精度受到桶边界影响，聚合 bucket 后算出来的是估计值，而不是每个请求精确 p95。

标签基数：为什么 request_id 不能成为 metrics label

Metrics 会按标签组合建立时间序列。若每个 request_id 都不同，时间序列数会随流量无限增长，造成内存、存储和查询压力。模型版本可用有限枚举标签；request_id、trace_id 放在日志/Trace 里，不放进高频 metric 标签。维度要控制在有限集合，例如 model_family、status_class、batch_bucket、device。

```
# Metric schema example; duration values use seconds.
requests_total{model_family, result_class}                # Counter
inflight_requests{model_family}                            # Gauge
queue_depth{queue_name}                                    # Gauge
runtime_calls_total{backend, result_class}                 # Counter
request_e2e_seconds_bucket{model_family, le}               # Histogram
batch_rows_bucket{model_family, le}                        # Histogram
```

把指标和前面的时间戳定义对齐

10.11 的 RequestTiming 已出现 submitted/enqueued/dequeued/bucketed/dispatched/runtime_begin/runtime_end/completed 这些事件；10.12 的任务是让每个 Histogram 确实读取正确的一对 timestamp，而不是换一层包装就算观测完成。先做纯函数 `duration_ms(start, end)`：缺失 endpoint 返回 optional/状态；end < start 拒绝；成功时验证单位换算；再把采样放在最终状态路径。

```
optional<double> elapsed_ms(optional<TimePoint> start,
                            optional<TimePoint> end) {
    if (!start || !end) return nullopt;   // unavailable, not zero milliseconds
    if (*end < *start) throw InvalidTimingOrder{};
    return duration<double, milli>(*end - *start).count();
}
```

Part B：TensorRT Engine 是编译产物，Context 才是一次执行的状态

可以把 TensorRT Builder 想成编译器：Network 描述计算图；Builder 根据目标 GPU、精度和输入范围试选 tactic；Config 放优化选项和 Profile；序列化得到 Engine/Plan。Engine 是可复用的已构建模型；ExecutionContext 是一次推理会话的执行状态，通常每个并行执行流/调用槽要有可独立使用的 context。不要每个请求都 build Engine。

```
ONNX / NetworkDefinition
          |
          v
Builder + BuilderConfig + OptimizationProfile
          | selects tactics, precision, and shape constraints
          v
Serialized Engine (plan) -> deserialize -> ICudaEngine
                                              |
                            createExecutionContext per execution slot
                                              |
                               set shapes + tensor addresses
                                              |
                                enqueue on CUDA stream
```

FP32 基线：为什么先做它？

FP32 基线先回答：导出的网络能不能构建和运行？输入/输出 shape、参数和 CPU/ORT reference 对不对？显存和稳定态延迟是多少？后续 FP16 的误差必须和同一输入、同一模型、同一 Runtime 路径的 FP32/参考结果比较。没有基线，FP16 更快或误差变大都缺少对照。

- 记录 ONNX SHA256、TensorRT 版本、GPU 型号/compute capability、驱动、构建参数、workspace/显存策略、Profile ranges。
- 分开 build time、deserialize/session startup、warmup、steady-state execution；冷构建和缓存命中不能混为一个 latency。
- 统一输入 shape、batch、dtype、warmup、repeats、同步方式；输出原始样本，不只写平均值。
- 校验 output shape、finite、max_abs_error、max_rel_error；误差定义写清楚近零分母处理。

10.12 实操：当天交付

工作时段：理论	工程时段：可操作步骤	验收/产物
1.5h 指标类型、单位与 histogram；1h 把 10.11 时间戳按区间画图	从已有 `TimingMs` 找每个 endpoint；写 metrics dictionary；实现 timing 单元函数；成功/失败路径都落一个终态样本	指标字段表、时序图、duration 单测；明确 serialization 尚未测
1h Engine/Context/Builder/Config 生命周期；半小时读 profile 对照	先确认 WSL 是否安装 TensorRT 并可 import；若没有，保存 Probe/错误，不盲改 PATH。可先实现 manifest/schema 与 mock 端到端构建接口	FP32 实验协议、环境 Probe、模型 SHA、构建日志与可复现命令

> 环境阻塞仍可推进：生成一份不可伪装成 TensorRT 实测的 FP32 任务清单/Manifest；但 Gate 的 build/test/pass 必须等到真实 TensorRT 工具链执行后再关闭。

10.13：Structured Logging + Trace + 精度路线

Part A：日志、指标、Trace 回答不同问题

信号	回答的问题	例子	不要拿它代替
Metric	整体是否退化？变化多大？	p95 request latency、拒绝率、队列深度	单个请求的完整故事
Structured log	某个事件发生时上下文是什么？	request_id、model_version、error_code、batch_id	大规模分布统计
Trace	这条请求在哪个阶段耗时/等待？	RPC→queue→batch→runtime→serialize span	无采样的总体计数

结构化日志不是把整句话写进字符串，而是把字段存为可检索的 key/value。固定 schema 能过滤和聚合；消息只补充人能读的上下文。`request_id` 是一次请求的服务内关联键；`trace_id` 跨进程/服务关联整条链；`span_id` 标识链中的一个工作区间。

一条请求如何被 Trace 连起来？

入口创建 server span，然后把 trace context 随请求对象传到 queue/worker/batcher/RuntimeAdapter。跨线程不能假设 thread-local 自动跟随：要在 request context 显式携带 trace context，worker 取出后恢复当前 span。Batching 时一个 batch span 可作为 batch 计算的工作区间；它关联多个 request，常用 span link 表达多对一因果，不要假装每个请求都拥有同一 parent-child 树结构。

```
Trace: request/trace-123
  server.request       [0ms ------------------------------ 42ms]
    admission          [1ms - 2ms]
    queue.wait         [2ms ----- 9ms]
    batch.wait         [9ms -- 13ms]
    runtime.infer      [13ms -------- 29ms]
    result.mapping     [29ms - 31ms]
    serialization      [31ms -------- 41ms]  # mark unavailable if not measured
```

日志字段建议与安全边界

- 固定字段：timestamp、severity、service、request_id、trace_id、span_id、model_version、backend、batch_id、queue_state、status_code、error_type。
- 错误路径记录 error class、错误阶段、可重试与否；避免把完整 Tensor 内容、用户 prompt、token、密钥或个人敏感信息写入日志。
- Trace 采样必须记录策略；高延迟/错误请求可提高采样，但不能只看采样后的 trace 推断总体 QPS 或 p99。
- 把 trace_id 放日志字段，不放 metrics label；低基数 `model_version` 是否允许进入 metric，要看版本数量和保留策略。

Part B：FP32、TF32、FP16、INT8 不是单纯的快慢档

精度选择是在算力、带宽、显存、支持算子和误差之间做权衡。存储格式决定权重/激活传输的大小；计算模式决定乘加内部使用的有效精度；TensorRT tactic 决定如何把 layer 映射到 GPU kernel。名称相似不表示误差机制相同。

模式	直觉	要测什么	常见误读
FP32	高精度基准和兼容基线	正确性、稳定延迟、显存、构建信息	把 reference 跑在不同预处理或不同输入上
TF32	对部分 FP32 矩阵运算使用较低乘法精度的硬件路径	是否启用、算子覆盖、输出误差和实际 speed	误以为 Tensor dtype 必然变成 TF32

模式	直觉	要测什么	常见误读
FP16	低精度浮点存储/计算路径，动态范围较 FP32 窄	max/mean abs/rel error、NaN/Inf、任务级质量、显存	只用一个随机小输入判定可用
INT8	整数化权重/激活；需 scale/zero point 或 calibration	校准数据、误差分布、质量、吞吐、兼容层	只编出 Engine 就代表精度可接受

精度 parity 的实验合同

```
for each precision in [fp32, fp16, int8_if_calibrated]:
    assert same_model_sha_and_input_set
    run reference + candidate on identical shapes
    report max_abs, mean_abs, max_rel, nonfinite_count
    report task-level metric if available
    report build_ms, startup_ms, steady_state_samples, peak_vram
    preserve raw output summary + environment + exact command
```

数值容差需针对算子、dtype 和模型给出理由。Relative error 在 reference 接近 0 时会爆大，所以同时看 absolute error，必要时用 `abs_error <= atol + rtol * abs(reference)`。实际部署的质量阈值取决于任务；讲义不能替模型定一个通用的 `1e-3` 就说合格。

10.13 实操：当天交付

- 给每个请求贯穿 trace_id/request_id；至少提供 server、queue、batch、runtime、response 五个 span/event。
- 跨 worker 显式传递 context；设计一个排队超时和一个 Runtime 错误样本，验证日志可从 trace_id 反查。
- 写 FP16 vs FP32 对照协议；若真实 TensorRT 阻塞，先对已有 PyTorch/ORT reference 跑离线数值统计框架，不把它称作 TensorRT parity。
- 产物：trace 示例 JSON、日志 schema、精度实验 manifest、环境/版本和模型 SHA。

> 一个很有用的诊断顺序：先看 Metric 确认总体症状，再用 trace 找慢阶段，最后用该请求的结构化日志解释状态/失败原因。逆序只盯日志，容易被偶然单例带偏。

10.14：过载保护 + G7 集成验收 + Dynamic Shape

Part A：过载保护的目标是让系统可预测地失败

当到达速率长期超过服务处理能力，队列会持续变长。没有界限时，系统把 overload 变成更长等待、更高内存、更多超时重试；重试又继续增加 offered load。过载保护的目标不是让每个请求都成功，而是在容量耗尽前保护延迟、内存和正在执行的请求，让拒绝快速而明确。

```
request -> validate -> admission gate -> bounded queue -> worker/batcher
             |             |                 |
             |             +-- reject busy   +-- absolute deadline check
             +-- reject invalid                    |
                                  runtime concurrency limit
                                             |
                                overload / failure-rate breaker
```

四种控制不要混为一谈

机制	限制什么	何时生效	失败语义
Queue capacity	等待内存/排队人数	入队时	队列满则拒绝或短时等待
Concurrency limit	同时在 Runtime 执行的数量	dispatch 前	保留请求但暂不运行/到 deadline 后过期
Rate limit	单位时间的请求/令牌数	入口 admission	按策略限速/拒绝；不是队列深度
Circuit breaker	对持续失败后端的调用	失败率/连续失败超阈值后	打开期间快速失败，半开探测恢复

这些门槛需要共同工作。队列有界不自动限制在途 Runtime 并发；并发有限不自动限制等待队列；rate limit 不保证 GPU 执行同时数；circuit breaker 不替代 deadline。给每个机制定义一个配置项、触发边界和测试。

过载时观察 offered / accepted / completed

Offered QPS 是客户端试图送入系统的速率；Accepted QPS 是 admission 接受的速率；Completed QPS 是完成的速率。三者拉开能定位入口拒绝、排队积压或执行能力瓶颈。若只报告 Completed QPS，就可能看起来容量稳定，但客户端有一半请求被拒绝。延迟也要把拒绝纳入结果分布或单列拒绝延迟，不能只对成功样本算 p95。

保护的分层逻辑

- 入口检查输入与模型版本，再做 admission。无效 shape 属请求错误，不应消耗队列和 Runtime 资源。
- 队列满时明确返回 overloaded/busy，不隐式无限等待。拒绝样本也记入计数与错误分布。
- worker dispatch 前再检查 absolute deadline 和 cancellation；排队取消要唤醒等待者并保证 future 完成一次。
- Runtime 调用中取消通常是 best effort：token/callback 能阻止后续映射，但不一定能撤销已提交到 GPU 的 kernel。
- Shutdown：停止接纳 -> 唤醒线程 -> drain 或按 policy fail remaining -> 完成每个 future 一次 -> join worker -> 释放 Runtime/Session。

Part B：G7 最小集成验收不能只有 happy path

测试	构造	要断言什么
Queue 满	容量设为 1，阻塞 worker，继续 submit	队列与内存有界；多余请求快速拒绝；拒绝率增加
慢 Runtime + deadline	FakeRuntime 可控延迟，短 deadline	过期请求不进入 infer 时 call_count=0；状态唯一
关闭时半满 batch	留下一部分请求后 close/drain	所有 accepted futures ready；每个只完成一次；线程 join 后销毁
Runtime 故障	inject exception / transient failure	错误映射准确；breaker 状态变化；请求不悬挂
批次隔离	同 shape 混入一个取消/过期项	有效行正确映射；无效请求不污染其他结果
过载恢复	逐步降低 offered rate	拒绝降低、Queue 回落、成功率恢复；观测信号一致

Part C：TensorRT Optimization Profile 是输入许可范围

若网络输入 shape 在 build 时有动态维度，例如 `[B,T,D]` 中 B 和 T 未知，就要用 runtime dimension placeholder，并在 build 时为每个动态输入建立 profile：min 是允许的最小形状，max 是允许的最大形状，opt 是希望引擎针对其优化的常见形状。profile 不是 profiler，也不是测量时长范围；它是输入 shape 合同加上优化器参考点。

```
TinyFFN input: [B, T, D], D=8 fixed
Profile 0:
  min = [1,   1, 8]
  opt = [1, 128, 8]   # expected common request
  max = [4, 512, 8]

Runtime path (conceptual, API names vary by TensorRT version):
  choose profile covering actual shape
  set concrete input dimensions on execution context
  verify dimensions are fully specified / infer output shape
  bind input and output addresses
  enqueue work on the chosen CUDA stream
```

opt 选得离实际流量很远，可能牺牲常见 shape 的优化质量；max 过宽会增加构建/内存代价，min/opt/max 也必须满足维度约束。越界形状应在进入 Runtime 前被拒绝或明确报错，不能等到异步 GPU 执行后才出现难定位错误。不同 profile 可能有不同资源占用；动态 shape 的 output 也可能需要先推导 shape，再配置/分配 buffer。

10.14 实操：当天交付

- 先实现并通过 FakeRuntime 故障注入矩阵：满队列、过期、取消、Runtime 错误、shutdown 与半满桶 drain。
- 测 accepted/completed/error/rejected QPS、queue_depth、inflight_requests、E2E histogram；明确样本口径。
- 用 TinyFFN 设计一个范围窄而有代表性的 profile；按 min/opt/max、常见输入和越界输入建测试表。
- 未在环境中真实运行 TensorRT 时，交付 Profile 设计和 blocked 环境记录，不写 Engine build passed。

10.15：Model Version / Hot Update + C++ Runner

Part A：为什么热更新不能简单覆盖一个全局指针？

一个服务正在用模型 v1 推理时，更新线程准备加载 v2。如果直接释放 v1 再把全局指针改成 v2，正在运行的请求可能还在访问 v1 的 Engine、Context、权重或 CUDA buffer，于是 use-after-free。安全切换需要把版本当作不可变的完整资源包，并让每个请求在开始时持有它的引用。

```
VersionBundle {
  version_id, model_sha, backend, Engine/Session,
  context_pool, allocator/buffer policy, readiness, metadata
}

new bundle: load -> validate -> warmup -> parity/smoke -> READY
switch:     atomic publish new immutable bundle for new requests
old bundle: old in-flight requests keep shared ownership
retire:     release only when no request/context/stream still uses it
rollback:   atomically point new requests back to previous READY bundle
```

一个请求何时确定 model_version？

在 admission 或 request dispatch 时固定版本句柄，整次推理从头到尾都使用这个 bundle。不要在请求中途重新读取 global current_version：否则前处理用 v1、batcher 按 v2 shape 分组、Runtime 调 v2、结果却按 v1 metadata 解释，会造成隐蔽错配。batch key 也需要包含版本/Engine identity，不能把不同模型权重的请求拼在一起。

Canary 与回滚

- 先加载 v2 到独立 bundle，做静态检查、shape smoke、数值 parity 与 warmup；失败时 v1 继续服务。
- 先把有限比例/受控请求路由到 v2，按版本比较错误率、latency、VRAM 和任务质量；标签需有限基数。
- 触发阈值后停止 v2 新流量，把 current pointer 切回 v1；v2 已在途请求可以完成或按明确 policy 终止。
- 记录切换事件：old_version/new_version、model SHA、actor、时间、原因、canary fraction、结果。不要在日志写模型输入内容。

```
auto current = std::atomic<std::shared_ptr<const VersionBundle>>{};

RequestHandle capture_for_request() {
    auto version = current.load(std::memory_order_acquire);
    if (!version || !version->ready()) return reject_unavailable();
    return RequestHandle{std::move(version)}; // held until request is terminal
}

void publish(std::shared_ptr<const VersionBundle> next) {
    validate_ready(next);
    current.store(std::move(next), std::memory_order_release);
}
```

这是概念示意，不是对现有 Inferflow 代码已经实现热更新的声称。C++ 标准库版本和原子 shared_ptr 支持情况要在工程目标中核对；也可用锁保护 shared_ptr 交换。关键合同是安全持有期、单请求版本一致性、并发交换和退役时不早释。

Part B：TensorRT C++ Runner 的 buffer 与 stream 生命周期

ExecutionContext 描述一次执行所需的 shape/profile/tensor address 状态；CUDA stream 是设备上按序提交工作的队列。enqueue 通常是异步提交：CPU 函数返回，并不表示 GPU 已算完。输入/输出 buffer 必须活到相关 stream 上的 GPU 操作完成；若调用方提前释放或复用，会变成数据竞争。

```
Host input
   | H2D copy (if input is host-resident)
   v
device input buffer -- TensorRT enqueue on stream --> device output
                                                        |
                              stream/event completion -+
                                                        v
                                             D2H copy if host output needed
```

Runner 该明确的资源契约

资源/动作	要回答的问题	常见错误
Device / CUDA context	这块指针属于哪张 GPU？	输入在 cuda:0，Context/stream 却在另一设备
ExecutionContext	是否被并发调用？shape/profile 状态是否隔离？	多个线程同时改一个 context 的动态 shape
Device buffers	谁分配、谁释放、何时可复用？	enqueue 返回后立刻 free 或覆盖输入
CUDA stream	copy、profile set、enqueue、D2H 是否在同一有序流或有 event 依赖？	跨 stream 缺依赖导致读到旧数据
Synchronization	何时需要 CPU 等待，测的是 host wall 还是 GPU event？	把 asynchronous enqueue 的函数耗时称作推理完成时间
Output shape	何时确定、谁分配动态输出？	按最大 shape 盲目分配，或输出 buffer 太小

推荐的执行阶段

```
1. Capture immutable model-version handle for this request.
2. Validate dtype, rank, shape range and device before GPU submission.
3. Acquire an execution context / buffer slot owned by this worker.
4. Select a profile that covers the input, set concrete dimensions.
5. Determine output shape; allocate/reuse appropriately sized buffers.
6. Set tensor addresses; enqueue on the owned CUDA stream.
7. Record completion event; keep all referenced buffers alive.
8. If host output is required, schedule D2H and wait only at boundary.
9. Map output to the original request, finish future, release slot safely.
```

TensorRT 的具体 API 在版本间会演进。例如当前文档以 name-based tensor address 与 enqueueV3 流程为主；旧 enqueueV2/binding-index 代码不能照抄到新接口。真实工程应先查当前安装的 TensorRT 文档和头文件，再写最小 build/execute smoke。下面的伪代码仅帮助掌握生命周期，不承诺可直接编译。

Host wall time 与 GPU event time

Host monotonic clock 包住 Runner 的完整调用，可以测 dispatch、内存分配、enqueue、等待和结果交付形成的 wall interval；GPU Event 则在某个 CUDA stream 上记录两个设备时间点，测它们之间流内经过的 GPU 工作。二者回答不同问题。若 CPU 只 enqueue 而不 synchronize，host wall 会显得很短；只有在结果完成边界同步后，这个请求的端到端 latency 才闭合。

10.15 实操：当天交付

- 画版本状态机：LOADING → VALIDATING → READY → ACTIVE → DRAINING → RETIRED；失败路径回到旧 ACTIVE。
- 用 Fake bundle 测热切换：v1 请求开始后发布 v2，检查旧请求仍拿 v1、新请求拿 v2；再制造 v2 加载失败并验证回滚。
- 写 C++ Runner 的接口草图和资源所有权表；如果 TensorRT 未装，不创建伪 Engine，也不写 TensorRT 测试通过。
- 一旦有 TensorRT 环境，先跑单线程 FP32 shape smoke，再做 context slot 多线程隔离和 stream/buffer 生命周期测试。

综合工程：把四天内容接回当前 Inferflow

将现有控制面（ServingEngine、Batching、FakeRuntime）当成验证骨架，不要一次性引入网络、Metrics SDK 和 TensorRT 三个新变量。建议沿以下阶段渐进完成，每个阶段保存 source identity 与独立测试结果。

阶段	实现/验证	关键断言	结果标签
A. Timing correctness	从 RequestTiming 派生每个 duration；补顺序/缺失/异常测试	无 end 不报 0；monotonic 顺序；终态只计一次	unit
B. Metrics	Counter/Gauge/Histogram exporter 或轻量 recorder	有限 label 集合；成功和拒绝分开；时间单位 seconds	synthetic/control-plane
C. Logs/Trace	RequestContext 带 request_id/trace context；跨 worker 显式传播	相同请求从 ingress 到 runtime 可关联；异常 span status	unit/synthetic
D. Overload	Bounded queue + max inflight + deadline + rejection	memory bounded；offered/accepted/completed 分开；future 单终态	synthetic
E. Version bundle	不可变 Runtime handle + atomic/locked publish	新旧在途请求绑定版本；load fail 不破坏 active model	unit/synthetic
F. Backend adapter	先 FakeRuntime，再 ORT CPU，之后 TensorRT/CUDA	同一接口、同一输入合同；记录 backend / model SHA	cpu_runtime/cuda_runtime
G. Capacity	Open-loop 与 closed-loop 分开扫描 concurrency/batch/shape	raw records、error/reject、queue、batch、compute 分阶段	synthetic 或真实后端明确区分

先执行 10.11 已编译的 synthetic sweep

当前已有 `benchmarks/inferflow/10.11_capacity_samples.csv`：2600 个 request rows，覆盖 T=16/64、max rows=1/4、closed-loop concurrency=1/4/8，每个 client 50 请求，DelayedRuntime sleep 3ms。12 组每组 status 均为 ok。源代码每组以 barrier 同时放行 client，每个 client 同步 future.get() 后才发下一请求，因此这是 closed-loop，不是固定到达速率。当前 CSV 逐 request 保存 Queue/pre-bucket/Batch Wait/Runtime/E2E，但没有 elapsed time、runtime_calls 和实际 batch-size 分布；因此可以分析时延样本，不能从 CSV 独立复算 QPS 或确认 batching efficiency。

```
Observation: replace this line with a value read from raw samples; do not guess.
Hypothesis: list a cause that can explain the observation and can be falsified.
Next test: hold T and max_rows fixed; change concurrency only; record each stage.

This is a template, not a measurement. Never infer QPS or batch counts from absent fields.
```

Open-loop 与 closed-loop 的对照实验

Closed-loop 通常是固定客户端数或 in-flight 数：一个客户端请求完成后再发下一个。系统变慢时，客户端自身发得也慢，存在 coordinated omission 风险，会低估过载下的尾延迟。Open-loop 则按固定时间表发请求，系统变慢仍继续到达，更能观察积压和拒绝；但要限制试验时长/总请求并保存客户端实际 send time 与 lateness。不同协议不能把 QPS/p95 混在同一行做“谁更快”。

控制	固定什么	系统变慢时	报告重点
Open-loop	预定到达速率/时刻	到达不自动减慢；Queue 会积压或拒绝	offered/accepted/completed QPS、client schedule lateness、拒绝与尾延迟
Closed-loop	并发客户端/在途目标	完成变慢后，下一请求也推迟，QPS 会降	实际 concurrency、完成 QPS、延迟及请求数

真实 synthetic CSV 的第一轮 Observation

T	max rows	concurrency	E2E p50/p95/p99 ms	Queue p95 ms
16	1	1	3.851 / 5.588 / 8.807	0.311
16	1	4	15.113 / 16.485 / 20.467	12.418
16	1	8	33.004 / 46.344 / 58.124	41.349
16	4	1	6.384 / 16.535 / 21.837	0.208
16	4	4	3.856 / 4.020 / 4.794	0.072
16	4	8	7.639 / 7.726 / 7.813	3.880
64	1	1	3.791 / 3.824 / 3.878	0.013
64	1	4	15.165 / 15.249 / 15.268	11.448
64	1	8	30.325 / 31.508 / 33.886	27.372
64	4	1	6.360 / 8.136 / 14.055	0.044
64	4	4	4.098 / 6.498 / 7.306	0.094
64	4	8	8.214 / 14.891 / 18.948	8.180

这些 percentile 按 source 中的 nearest-rank 规则从每组成功 E2E 样本算出（ceil(p*n)-1），单位为毫秒。CSV 是 2600 行、12 组；每组样本数对应 concurrency×50：50、200、400。所有保存状态都是 ok。Runtime p50 多数约 3.76-3.79ms，略大于代码请求的 3ms sleep，这来自 sleep_for/调度及计时边界，不是纯 GPU kernel。

从数据可以安全说什么？

- max_batch_rows=1 时 concurrency 从 1 到 4/8，E2E 与 Queue p95 大幅上升；在这个单 worker/single-runtime 结构与 closed-loop client 模型下，观察到排队等待增长。
- max_batch_rows=4 的 concurrency=4，E2E 分位数明显低于 rows=1 对应配置；这是与合批设置相关的相关性，不足以证明 runtime_calls 减少多少，因为 CSV 没保存 runtime_calls 或实际 batch histogram。
- concurrency=1 且 max rows=4 时，batch wait/timeout 会让 p50 高于单行场景；允许更大 batch 不代表每个请求必然加入满批次。
- E2E 分位数和 Queue 分位数不应相减当作 batch/compute 的分位数。分位数不具备线性可加性，而且各 interval 样本可能相关；逐请求先计算 stage durations，再分阶段聚合。

Observation / Hypothesis / Next test

```
Observation:
  T=16, max_rows=1: concurrency 1->8 changed E2E p95 5.588->58.124ms.
  Queue p95 changed 0.311->41.349ms; all 12 groups report status=ok.

Hypothesis:
  Queue wait may dominate the increase. Batching benefit is unproven because
  runtime_calls and actual batch sizes are absent from this CSV.

Next test:
  Persist elapsed, runtime_calls, batch-size histogram, accepted/completed counts.
  Interleave max_rows=1/4 at fixed T and concurrency; repeat at least five rounds.
```

工程验收命令模板

以下是工作流模板，路径和 target 名要以当前仓库的 CMakeLists.txt 为准。先确保命令落在实际项目目录，构建目录仅放 build artifacts；不要把 CMake build dir 加入 Git。

```
# From the repository root; confirm the actual target and output arguments first.
cmake -S cpp -B cpp/build-mac -DBUILD_CUDA_LAB=OFF \
  -DBUILD_TESTING=ON -DBUILD_INFERFLOW_BENCHMARKS=ON
cmake --build cpp/build-mac --target inferflow_capacity_sweep -j
ctest --test-dir cpp/build-mac --output-on-failure -R 'inferflow'

# Use the executable usage for CSV output. Save git revision/status and exact command.
```

> 10.11 的 target 只证明确实编译成功。上面的命令是建议的下一步，不代表已经执行；先用 `cmake --build ... --target help` 和可执行文件 `--help`/usage 核对 target/参数，避免猜测命令。

四天验收：闭卷、实现、证据三条线

闭卷问题（不看前文先答）

- Counter、Gauge、Histogram 分别适合记录哪三个 Serving 量？为什么 request_id 不该放进 metric label？
- queue_time、pre_bucket_wait、batch_wait、runtime_time、E2E 的起止事件分别是什么？serialization 没采集如何表示？
- 为什么只对成功请求算 p95 会让过载看起来更健康？offered/accepted/completed QPS 各说明什么？
- Trace ID、Span ID、Request ID 的关系是什么？跨 worker 时为什么不能依赖 thread-local 自动传递？
- Queue capacity、max inflight、rate limiter、circuit breaker 分别限制什么？谁能保证内存有界？
- Shutdown 时为什么先 stop admission，再 drain/settle future，然后 join，最后释放 Session？
- FP32、TF32、FP16、INT8 的数据格式/计算路径差异如何反映在 parity 报告中？
- Optimization Profile 中 min/opt/max 分别是什么？opt 不是 profiler 的 measurement interval，为什么？
- Engine 与 ExecutionContext 各自承载什么？为什么不能每个请求重新 build Engine？
- 异步 enqueue 返回时 GPU 是否一定完成？Input/output buffer 最短需要活到什么时候？
- 模型热更新时，旧请求怎样安全完成？batch key 为什么要包含 model version/engine identity？
- closed-loop concurrency 扫描与 open-loop QPS 扫描，在过载时的偏差有何不同？

建议评分尺

维度	达标标准	证据
概念	至少 10/12 正确，且指标边界、profile、版本生命周期三类无混淆	闭卷原答 + 订正
Serving 工程	Queue/Batch/Runtime/E2E 的 timestamp 通过纯函数/路径测试；过载关闭和 drain 不悬挂	CTest 原始输出 + failure injection
Observability	能用一次 trace 解释慢请求，metrics label 有有限 cardinality	Trace JSON / metrics snapshot
TensorRT	Probe/build/execute/parity 至少一条真实路径；若环境阻塞，Gate 如实 blocked	版本、GPU、build log、engine SHA、数值和 latency raw data
实验结论	O/H/N 与 raw CSV 对应；synthetic、cpu_runtime、cuda_runtime 标签准确	run manifest、raw samples、复现命令

每日时间切片（总计约 30 小时）

日期	理论 2.5h	工程 4.5h	收尾 0.5h
10.12	指标类型、指标边界、Engine/Context	Timing 单测 + metrics schema + FP32 环境/协议	保存 probe 与 manifest
10.13	Structured logging、Trace、精度格式	trace context 贯穿 worker + FP16 parity harness	写错误样本和 limitations
10.14	过载、Admission、Profile 范围	故障注入 + G7 integration + shape boundary	汇总 pass/fail/blocked
10.15	版本状态机、Runner/Stream/Buffer	hot-swap fake tests + TensorRT Runner skeleton/真实 smoke	版本/OH/N 与回滚记录

计划中的理论工程占比可按实际工作日调整，但不要把每日追赶槽压缩掉。若 TensorRT 环境缺失，先完成理论、接口合同、Mock lifecycle 和环境 Probe；阻塞证据保留，随后续计划继续追赶。

如何写日报中的最终结论

```
topic: Serving / TensorRT
completed: concepts read, code changed, commands actually executed
theory_score: raw score and missed concepts
tests: exact command, pass/fail/skip, environment (separate build from test)
metrics_trace: timing boundaries, raw samples, sampling policy
tensorrt: version, GPU, model SHA, profile, precision, engine SHA
observation: directly measurable facts only
hypothesis: falsifiable explanation
next_test: vary one major factor and state expected distinction
blocked: environment/dependency/code layer and saved raw error
```

深度辅导 A：Metrics 从时间戳到仪表盘，一步一步做对

前面建立了指标类型和字段表；本节重点是把一个请求的原始事件，可靠地变成可以长期比较的 metrics。你可以把它想成测量一条赛道：先决定起跑线、终点线、计时器和分组方法，再比较运动员。只写 `latency` 没说清是哪两条线。

A1. 先把事件点定义成状态机，而不是零散变量

请求的状态应有单向推进关系，例如 RECEIVED → ADMITTED → QUEUED → DEQUEUED → BUCKETED → DISPATCHED → RUNNING → RESULT_READY → COMPLETED。拒绝、取消、deadline、runtime error 是终止分支，不是成功路径上稍后再补的一个布尔值。每个状态进入时间最多写一次；重复进入要有明确语义，否则同一请求可能被计时两次。

```
enum class RequestState {
  Received, Admitted, Queued, Dequeued, Bucketed, Dispatched,
  Running, ResultReady, Serialized, Completed, Rejected, Expired,
  Cancelled, Failed
};

// transition() should reject impossible transitions and duplicate terminal states.
bool transition(RequestContext& r, RequestState next, TimePoint now) {
  if (is_terminal(r.state)) return false;
  if (!allowed(r.state, next)) return false;
  if (r.timestamps.contains(next)) return false;
  r.timestamps[next] = now;
  r.state = next;
  return true;
}
```

这里的核心不是一定要用 map 或 enum；而是把状态和时间点绑定。若请求在 admission 就被拒绝，它没有 enqueued/dequeued 时间，Queue Time 应为 unavailable，不应伪造 0。若 Runtime 失败，它仍然有 runtime_begin/runtime_end 和 E2E，只是 status 不成功。

A2. 用一条具体请求手算每个 latency

事件	时间 ms	解释
submitted	0	客户端进入服务计时边界
enqueued	1	admission 通过并进入有界队列
dequeued	4	worker 取出请求，Queue Time 结束
bucketed	6	请求加入与 shape/version 匹配的桶
dispatched	10	形成 batch 并开始 Runtime 交付
runtime_begin/end	10 / 16	IRuntime::infer() 的 host wall interval
result_ready	17	结果映射完成
serialized	20	结果编码完成
completed	21	响应交给传输层/完成 future

据此 Queue Time=4-1=3ms，dequeue 后 pre-bucket=6-4=2ms，Batch Wait=10-6=4ms，Runtime=16-10=6ms，serialization=20-17=3ms，E2E=21-0=21ms。E2E 并不一定等于这些值相加：例子里 submitted→enqueued 的 admission 1ms、result mapping 1ms、serialized→completed 1ms 也在 E2E 内。先定义全部段，再决定哪些段对外暴露。

> 不要用 p95(E2E) - p95(Runtime) 推出 p95(Queue)。p95 是排序后的样本位置，不满足可加减。若要解释尾请求，先对同一个 request_id 算各阶段 duration，再分别看阶段分布，或者选 p95 request 追 Trace。。

A3. Histogram 桶怎么定？

Histogram 要能回答业务 SLO。例如目标是 p95 < 100ms，就应在 100ms 附近有足够细的边界；如果所有桶都在 1s、10s、60s，100ms 的尾部只能得到很粗估计。另一方面，桶无限细也会增加时间序列/存储成本。常用做法是围绕延迟范围按倍数增长，靠近 SLO 与 batch trigger 的地方加密。记录单位固定为 seconds，展示时再格式化成 ms。

```
Example bucket boundaries in seconds:
0.001, 0.002, 0.005, 0.010, 0.020, 0.050,
0.100, 0.150, 0.200, 0.300, 0.500, 1.0, 2.5, +Inf

Choose from observed service scale and SLO; these are examples, not universal defaults.
```

Prometheus histogram 的每个 `le` bucket 是累计计数：le=0.1 的 count 包含所有小于等于 100ms 的观测。p95 需要跨桶插值，因此只有 bucket 边界之间的近似。经典 histogram 可以跨实例合并 bucket 后估算 quantile；Summary 的客户端预计算 quantile 通常不能像 bucket count 一样任意跨实例合并。要在配置里保存 metric type 和 bucket list，避免不同版本的服务偷偷改变统计口径。

A4. Cardinality 用乘法估算

假设你给某个 histogram 配了 `model_version` 12 种值、`status` 5 种、`device` 2 种、`route` 8 种，每个 histogram 有 14 个 bucket，加上 sum/count 等内部序列，粗略组合会随标签笛卡尔积增长：12×5×2×8×14=13,440 条序列，单个指标就可能如此。再乘几十个服务副本和多种 metrics，代价会非常大。`request_id` 有无限取值，不能当标签。

A5. 四类 QPS 口径要共用一个时间窗

Offered 是客户端尝试送进来的请求数/秒；Accepted 是 admission 接受的请求/秒；Completed 是已走到终态的 accepted 请求/秒；Successful 是成功返回请求/秒。拒绝、超时、取消也要单列。所有分母要采用相同 wall-clock window；若只对 accepted 的时间窗计算 completed，两个速率不能直接比较。Closed-loop 测试里 offered 也不是固定到达速率，应该称 offered request count 或 observed submit rate，并公开客户端模型。

A6. 指标采样的测试模板

```
test: admitted request records event times
  expect queue_ms = t_dequeued - t_enqueued
  expect pre_bucket_ms = t_bucketed - t_dequeued
  expect batch_wait_ms = t_dispatched - t_bucketed
  expect runtime_ms = t_runtime_end - t_runtime_begin
  expect e2e_ms = t_completed - t_submitted

test: rejected before enqueue
  expect queue_ms = unavailable (not 0)
  expect request outcome counter rejected += 1 exactly once

test: runtime throws
  expect runtime_ms exists, status=runtime_error, completed once

test: timestamp missing / reverse ordered
  expect validation error or unavailable field; never negative latency
```

完成这些测试后，Metrics 接入才有可信基础。否则一个字段拼错或漏打点，会变成长期 dashboard 上看似稳定的错误事实。

深度辅导 B：日志与 Trace 让异步请求仍然可追踪

B1. 三个 ID 分别是什么？

request_id 是服务 API 里的业务/请求标识；trace_id 是跨服务的一次因果链标识；span_id 是该链中的一个操作区间。一个 trace 可以含多个 request（例如 fan-out）；一个 batch infer span 也可能同时服务多个 request。因此这些 ID 常常有关联，但不能假设永远一对一。

字段	生命周期	建议出现位置	标签策略
request_id	一个逻辑服务请求	request context、日志、trace attribute	禁止作为 metric label
trace_id	跨服务/进程的一条调用因果链	入口提取/创建，跨 RPC 传播，logs/traces	不能作为 metric label
span_id	一个 operation/span	每段 trace 与相关日志	不能作为 metric label
batch_id	一次组 batch/一次 Runtime 调用	batch span、Runtime log、BatchSlice 映射	不要按唯一 batch id 创建 metric series
model_version	一个不可变 model bundle	每条请求日志/trace；低基数 metrics 可用	数量应受控，退休版本要清理

B2. Structured Logging 的 JSON schema

将日志做成 key/value，而非拼一条只适合人眼的字符串。字段可被查询系统解析、筛选、聚合和关联；固定命名还能减少一段代码写 `requestId`、另一段写 `req_id` 的 schema 漂移。每一条日志不要把整份 RequestContext 或 Tensor dump 出来，只记录诊断所需的安全元数据。

```
{
  "ts": "2026-10-15T08:30:01.123Z",
  "severity": "WARN",
  "service": "inferflow",
  "event": "request_rejected",
  "request_id": "r-0182",
  "trace_id": "...",
  "span_id": "...",
  "model_version": "tinyffn-v7",
  "backend": "fake",
  "shape": [1, 256, 8],
  "queue_depth": 64,
  "status_code": "RESOURCE_EXHAUSTED",
  "reason": "admission_queue_full"
}
```

时间戳用于跨系统粗略排序，Duration 仍优先由同一进程的 monotonic clock 计算。Wall clock 会被校时/时区影响，不能拿系统时间的差值当精细 latency。Trace 的 start/end 通常也是统一的时间表示，但 Instrumentation SDK 会处理时钟/上下文；不要手工把两台主机时间相减得出精确网络耗时。

B3. Trace 如何跨线程？

同步函数调用时，SDK 可能用 thread-local 保存当前 span；但请求离开 RPC 线程进入 C++ worker queue 后，执行线程变了，thread-local 不会神奇地携带原 trace。应在 enqueue 时把 SpanContext/TraceContext 放到 RequestContext，worker dequeue 后从 context 恢复父关系，再开 batch/runtime 子 span。若线程池会重用线程，结束 span 后必须恢复/清理当前上下文，防止下一个请求继承错误 trace。

```
Ingress thread: extract incoming context -> create SERVER span
                        |
                        +-- RequestContext.trace_context = current context
                        |
Queue push: copy immutable trace context with request
                        |
Worker thread: restore parent context -> create queue/batch/runtime span
                        |
Batch spans: one operation may relate to N request spans; use links where appropriate
```

B4. 一个请求排队慢时如何诊断？

- Metric 先指出 `request_e2e_seconds` p95 上升，同时 `queue_depth` 和 `queue_wait_seconds` 是否同步上升。
- 选一个慢请求的 trace_id，在 trace UI 看完整 span timeline；确认耗时落在 admission、queue、batch wait、runtime 还是 serialization。
- 根据该 span 的 request_id 关联结构化日志，检查 shape、model_version、deadline、拒绝原因、batch_id 与 error class。
- 对照并发/到达速率和运行版本；先形成假设，再单独改变 max_batch_rows、Wmax 或 worker count 验证。
- 不要用一个 trace 的结果推算总体分布，也不要只对 sampled successful requests 声称 p99 改善。

B5. Batch 的一对多关系

Batch 合并多个请求后，Runtime 看见的是一个大 Tensor。若记录 trace，父请求 A/B/C 各自已存在独立的 request span；一个 batch execution span 由调度器创建，可用 span links 指向多个父请求上下文。结果切片后，每个 request span 分别记录自己的结束与结果。将 A/B/C 都塞成同一个 parent 的 child 会错误表达真实因果关系，尤其当它们属于不同 trace 时。

B6. Trace 采样与隐私

全量 Trace 会有成本，通常按概率或 tail-based policy 采样。低延迟正常请求可以低比例采样；错误、超时、超长请求可以优先保留，但需要明确 SDK/exporter 是否支持 tail sampling，并设置内存上限。采样改变了被观察样本的组成：如果系统专门保留慢请求，那么 trace latency 样本不能再当总体分布。敏感 prompt、token、用户 ID 默认不写入 attribute，也不放进 baggage；外部传入的 trace header 需要长度与格式校验。

> 检查清单：Trace ID 入口确定；RequestContext 显式携带；跨 worker恢复；Queue / Batch / Runtime / Response span边界和 metrics 一致；异常设置错误状态；采样和敏感信息策略可说明。

深度辅导 C：过载保护与优雅关闭，不只是 if(queue_full)

C1. 为什么过载时延会突然变坏？

把 Runtime 想成一台每秒大约能做 250 次操作的机器，入口却持续送 240 次/秒。利用率已经很高，一点点服务时间抖动、GC/分配、锁竞争，就会让队列等待剧烈变化。若入口稳定送 300 次/秒，排队工作量持续净增加，任何有限内存最终都会耗尽。服务需要 admission/backpressure，不应把无穷等待伪装成“请求还在处理”。

简化的 M/M/1 排队模型有 ρ=λ/μ；当 ρ 接近 1 时，平均系统时间按 `1/(μ-λ)` 非线性增大。真实 GPU Serving 通常有批处理、并行 worker、非指数到达和 shape 混合，不能拿这个式子当真实预测；它只是直觉工具：剩余容量越少，抖动越容易放大成排队。

C2. 一个请求至少经过四道控制门

门	检查对象	例子	失败发生在哪里
输入校验	rank/dtype/shape/size/model version	shape 过 profile max	入口快速 INVALID_ARGUMENT
Admission	服务是否还收请求、队列容量是否有空间	queue size=capacity	RESOURCE_EXHAUSTED/OVERLOADED
Dispatch gate	deadline、取消、backend 并发 slot	deadline 已过/无 slot	不调用 Runtime，完成 future
Runtime guard	后端当前可用/熔断状态/模型 ready	backend 连续失败，breaker OPEN	UNAVAILABLE 或映射 Runtime error

Admission 应尽早拒绝，避免把明显不能执行的请求塞进 queue；但真正执行前仍需第二次检查 deadline/cancel，因为请求会在等待期间失效。过期判断使用绝对 monotonic deadline，把剩余预算向下游传播，而不是每一层都重新给完整 timeout。

C3. Queue 有界和并发有界不是一回事

假设 queue capacity=128，worker queue 不会无限长；但若有 64 个 worker，每个都持有巨大 Tensor/Context，内存仍可能很大。反过来，GPU runtime 只允许 2 个并发，但 queue 没上限，剩余请求仍会在主机内存累积。完整内存预算可粗略写成：`queued request count × max bytes/request + in-flight slots × input/output/workspace bytes + batch buffers + model/context memory`。batcher 自己的 bucket 和 deferred list 也要纳入容量。

因此每个资源池都要明确容量：ingress pending、queue、deferred、batch bucket、context pool、device buffer pool。每个等待点都要有 deadline 或可取消退出路径。`std::vector` 动态增长本身不是 backpressure。

C4. Circuit breaker 状态机

```
CLOSED: requests flow; failures counted in a rolling window
  if failure threshold crossed -> OPEN
OPEN: fail fast; do not send more traffic to unhealthy backend
  after cool-down -> HALF_OPEN
HALF_OPEN: allow a small number of probes
  probe success -> CLOSED; probe failure -> OPEN again
```

Circuit breaker 用来隔离持续失败的后端，防止每个请求都消耗满一个长 timeout 并拖垮 worker；它并不限制每秒请求数，也不保证队列长度。统计窗口要区分网络错误、bad input、用户取消和模型 bug：如果将 INVALID_ARGUMENT 算作 backend failure，可能错误打开 breaker。Half-open 探测要限量，避免恢复瞬间涌入全量请求。

C5. Deadline 与 Cancellation 的两个时刻

一个 request 可在入队前已经过期；也可能进入 queue 时有效、等到 worker 准备 dispatch 时才过期。两处都需要检查。若过期发生在 enqueue 前，queue/runtime metrics 不存在；若发生在 dequeue 后、dispatch 前，queue 已结束，pre-bucket/batch wait 会增加，并且 Runtime call_count 必须保持 0。Runtime 一旦把 GPU kernel enqueue 到 stream，取消通常不能保证硬中止；可以阻止后续工作、屏蔽结果或做 best-effort cancellation，但必须说明真实语义。

```
submit(req, absolute_deadline):
  if expired(now, deadline): complete(DEADLINE_EXCEEDED); return
  if !admission.try_accept(req): complete(RESOURCE_EXHAUSTED); return
  queue.push(req)

worker(req):
  if req.cancelled(): complete(CANCELLED); return
  if expired(now, req.deadline): complete(DEADLINE_EXCEEDED); return
  if !runtime_slot.acquire_until(req.deadline): complete(DEADLINE_EXCEEDED); return
  call runtime once; map result; complete exactly once
```

C6. 关闭顺序是一条所有权链

典型 drain：一、把状态改为 Closing，拒绝新 submit；二、唤醒在 condition_variable 上睡眠的 worker；三、按策略 drain queue/deferred/bucket，或对剩余请求设置明确终态；四、等待正在执行的 Runtime 调用完成，settle 每个 accepted promise 恰好一次；五、join 所有 worker；六、确认没有 stream 仍使用 buffer，再释放 context/engine。`close()` 返回后仍有 future 悬挂，或释放 Engine 时 CUDA 还在读其资源，都是生命周期 bug。

C7. 故障注入矩阵如何写

场景	可控手段	必须断言
队列满	worker barrier 暂停 + capacity=1	新请求快速拒绝；队列 size <= cap；拒绝 metric +1
过期排队	Runtime gate 慢放行 + deadline	dispatch 前过期；runtime calls=0；future ready
取消排队	promise 保持待取，随后 cancel	worker/queue 被唤醒；终态 CANCELLED 一次
Runtime throw	FakeRuntime 注入异常	error mapping 稳定；worker 仍能处理下一请求
close 半批	凑不到 max rows 即 close	policy 明确 flush 或 fail；所有 future 结束后 join
重复 shutdown	并发调用 close/stop	幂等；不 double join、不 double fulfill、不 use-after-free

为避免时序测试偶发，尽可能用 latch/barrier/condition_variable 控制执行点，不要靠 `sleep_for(50ms)` 猜线程已经运行。测试结束要释放 gate、join thread，并设置总 timeout 防止 pytest/CTest 永久挂住。

深度辅导 D：模型版本切换从对象图到失败回滚

D1. 一个版本不是一个文件路径

真正可执行的版本对象是一组互相匹配的资源：model/plan SHA、输入输出 schema、precision、优化 profile、ExecutionContext 池、buffer allocator 策略、backend/device、warmup 状态和可观测元数据。只更新 `model_path` 而不更新输出 shape/schema/cache，会出现旧 metadata 解读新结果的错配。把 VersionBundle 当作不可变对象，构建完成后不再原地修改。

```
struct VersionBundle {
  VersionId id;
  Sha256 model_sha;
  InputContract input_contract;
  OutputContract output_contract;
  Precision precision;
  std::shared_ptr<Engine> engine;
  std::shared_ptr<ContextPool> contexts;
  std::shared_ptr<BufferPool> buffers;
  std::atomic<Readiness> readiness;
};

struct RequestHandle {
  std::shared_ptr<const VersionBundle> pinned_version;
  RequestId id;
};
```

D2. 为什么请求入口固定版本，而不是 Runtime 调用时才读 current？

请求可能经历较长 queue/batch wait。若在 submit 时接受请求但不记录版本，期间版本 pointer 被切换，请求在 delay 之后可能突然改用新版本，这会让结果无法重现，也会让 batcher 把旧/新版本放进同一 batch。固定版本 handle 的好处是从接受开始就有稳定语义：此请求由 v1 负责，下一批新请求才由 v2 负责。

对于 dynamic batching，batch key 至少要包含 model/version identity、输入 dtype、shape bucket/profile、精度和可能的 adapter configuration。只有这些兼容的请求才能沿 batch 维拼接。相同 shape 并不代表可以混用不同 weights。

D3. 热切换状态机与不可逆点

状态	动作	允许接收请求吗？	失败怎么办？
LOADING	读权重/Engine/Plan 与依赖	否	丢弃 candidate，旧 active 不变
VALIDATING	schema、shape、device、SHA、安全检查	否	标记 candidate failed
WARMING	warmup + parity + 资源预热	否	不 publish；保存错误
READY	健康检查和资源池可用	候选可切换	等待 canary/发布命令
ACTIVE	新请求 capture version handle	是	新版本失败时发布旧 READY 版本
DRAINING	等待旧版在途 request/context/stream 完成	不接新请求	到 drain timeout 后按 policy 处理
RETIRED	确认无引用后释放 GPU 资源	否	不能再回滚到已释放版本

不可逆点是旧版本已释放资源。回滚如果仍可能发生，就不能过早 retire v1；至少保留上一版 Engine 或能快速加载回来的可靠来源。GPU VRAM 不足时，需要计划切换过程的双份显存峰值：同时有 v1/v2 Engine、workspace、buffers/context pool 时，可能比稳态高很多。

D4. 热更新测试逐条演练

- old request captured v1 后暂停在 Runtime 前；publish v2；释放执行门，断言该请求仍从 v1 得到结果。
- publish v2 后 submit 新请求，断言它 pin v2；BatchKey 把 v1 与 v2 分开。
- 让 v2 校验失败、warmup 失败、context pool 创建失败，active pointer 始终是健康 v1。
- 并发多个 reader 读取 current，同时 writer 反复发布；每个请求观察到的 handle 不为空且不可变。
- 在 v1 仍有 in-flight request 时启动 drain；确认 Engine/Context 延后释放，所有 future 完成后再 retire。
- 模拟 canary 指标超阈值；回滚到 v1，记录 v2 新请求停止、在途请求策略以及切换事件日志。

D5. 双版本 Canary 的指标设计

Canary 最重要的是分母明确：v1/v2 各接受多少请求、成功多少、拒绝/超时多少；比较的是同样 shape 与负载下的 p95、显存、错误率以及任务质量。若 v2 只拿到短 prompt、v1 拿到长 prompt，latency 差异不能归因于版本。按模型版本分 metric 前确认版本标签有限，不要每次 commit SHA 形成无限 series；完整 SHA 放日志/manifest。

```
rollout record:
  old_version, new_version, old_model_sha, new_model_sha
  rollout_fraction, start/end, canary_sample_count
  parity/status, e2e p50/p95/p99, error/reject rate, peak VRAM
  decision = continue | pause | rollback
  decision_reason + exact build/config identity
```

> 热更新验收的重点不是 pointer store 是否原子，而是 request 看到的完整资源图在生命周期内保持一致。`atomic<shared_ptr>` 只能帮助安全交换所有权；它不能自动保证 CUDA work 已完成、Context 可并发使用或显存预算充足。

深度辅导 E：TensorRT 从网络定义到可重复 Engine

E1. 把 Builder 当成针对目标硬件的编译器

ONNX 是图的交换格式，不是最终 GPU 程序。TensorRT 解析网络、做合法化与融合、根据设备和约束枚举 layer tactic，再把选择组合成 Engine。Build 可能花几秒到更久，因为优化器在试运行/测量候选实现；推理时则 deserialize Engine，创建 ExecutionContext，配置 shape/profile 和地址，再 enqueue。Build time 与 inference latency 是完全不同指标。

阶段	输入/对象	产物	容易漏的验证
Parse/Network	ONNX、weights、network definition	network layers/tensors	parser error、动态维度和算子支持
Build	Builder + BuilderConfig + profiles + GPU	serialized plan/Engine	precision flags、workspace、tactic/build log
Deserialize	plan bytes + runtime	ICudaEngine	版本兼容、GPU兼容、plan完整性
Create context	Engine + execution slot	IExecutionContext	并发使用策略、profile ownership
Execute	shape/address/stream	output buffer contents	异步完成、地址有效性、错误检查

Engine 通常包含针对构建约束选择好的执行计划；不要把它当成已经执行的输出 Tensor。ExecutionContext 持有运行期可变状态。单个 context 若被多个线程并发修改 shape/address/profile，可能互相覆盖；常见安全设计是每个并发 slot 独占一个 context，或在严格串行锁下复用。

E2. Engine fingerprint 应记录什么？

- source model SHA256、模型导出参数、输入输出 schema、动态 shape Profile 列表。
- TensorRT/CUDA/cuDNN/driver 版本、GPU 型号与 compute capability、OS/架构。
- Builder flags、precision、workspace/memory pool、tactic sources、optimization level。
- Engine/plan 文件 SHA、size、mtime、build duration、build logs、是否命中 timing cache。
- 成功/失败状态、parser/build error、保存路径及 exact command；失败也要落 manifest。

Timing Cache 帮助跨 build 复用 tactic timing 信息，但 cache 不是 Engine；缓存匹配程度受硬件、版本、配置影响。cold build 与 warm build 应固定 source identity 和参数，只变化 cache 状态，且不能把一次 cache 命中当成推理性能提升。

E3. FP32 输出基线怎么构造？

固定一组测试输入，覆盖常见和边界 shape；使用同一模型参数和同样预处理，让 PyTorch 或已有 ORT 输出作为 reference。FP32 TensorRT 结果与 reference 的差异应先在正确性可接受范围内，然后再建立 latency 基线。输出有多维时，不能只比第一行或均值；检查每个 output tensor 的 shape、dtype、finite、最大误差和抽样位置。

E4. Engine 缓存误用案例

缓存目录里存在 `model.plan` 不等于该 plan 对应当前 ONNX。若只用文件名作为 cache key，修改模型权重或 profile 后仍可能加载旧 Engine。安全 key 至少依赖模型内容 SHA、TRT 版本、GPU compatibility 条件、builder config、profiles、precision 与 relevant plugin versions。load 时把 manifest 与 Engine fingerprint 核对，不匹配就拒绝加载或重新 build。

```
engine_cache_key = SHA256(
    model_sha + tensorrt_version + gpu_target + builder_flags
    + profile_contract + precision + plugin_versions
)

load plan only if manifest.cache_key == expected_cache_key
and plan hash matches manifest.plan_sha256
otherwise mark stale and rebuild (if allowed)
```

E5. Build Error 的分层诊断

看到的阶段	可能问题	下一项证据
import/runtime missing	Python 环境/库路径/安装包	python path、TensorRT version、ldd/loader
ONNX parse fails	模型 opset/op support/shape metadata	parser error line/node/opset/model SHA
build fails	workspace/profile/tactic/unsupported precision	完整 build log、GPU、flags、profile min/opt/max
deserialize fails	版本/硬件/plan损坏/插件缺失	Engine SHA、build env、plugin registry、runtime version
enqueue fails	shape/address/profile/stream mismatch	逐 tensor 地址、当前 shape、profile、CUDA error
wrong output	预处理/layout/dtype/buffer race/数值精度	reference parity、buffer lifetime、stream event timeline

深度辅导 F：精度模式与 Dynamic Shape 的手算训练

F1. 精度至少有三个层次

讨论 precision 时分清：一、Tensor 的存储 dtype（FP32、FP16、INT8）；二、算子乘法/累加使用的有效精度与硬件路径；三、Engine Builder 是否允许选择低精度 tactic。用户看到输入 Python array 是 float32，不代表所有内部计算严格 IEEE FP32；反过来，开启 TF32 也不一定把整个模型输入输出 Tensor dtype 改成 TF32。具体行为与 Runtime 版本、硬件和算子有关，要看实际配置/Kernel。

F2. 误差公式如何读？

```
abs_err_i = abs(y_candidate_i - y_reference_i)
rel_err_i = abs_err_i / max(abs(y_reference_i), epsilon)

pass element i if:
  abs_err_i <= atol + rtol * abs(y_reference_i)

always also report:
  max_abs, mean_abs, max_rel (with near-zero policy), nonfinite_count
```

Atol 对接近 0 的 reference 更有意义；rtol 按 reference 大小缩放。不能只报告“torch.allclose passed”，因为阈值若不写、dtype/shape 不同、输入不同，pass 的信息量很弱。若输出有 1e6 个元素，单个坏值可能被 mean error 稀释，因此至少报告 max、分位数、nonfinite 和失败元素索引。

F3. 一组具体误差例子

假设 reference 输出是 [100.0, 0.01, 0.0]，candidate 是 [100.2, 0.011, 0.002]。绝对误差是 [0.2, 0.001, 0.002]；相对误差约 [0.2%, 10%, 无穷/需 epsilon 策略]。一个仅看 max relative error 的表会突出近零噪声；一个仅看 max absolute error 的表会忽略 0.01 被偏成 0.011 的比例变化。两者必须共同解释，并结合模型任务质量。

F4. FP16 与 INT8 的风险不是同一个

FP16 是浮点格式，动态范围小于 FP32，某些大值可能溢出、很小值可能下溢；累加策略也要关注。INT8 是量化整数值，需 scale/zero-point 或其他量化方案把实数映射到有限离散级别。Calibration 的任务是从代表性样本估计激活范围/量化参数；校准数据分布如果偏离线上输入，错误可能不是在随机 unit tensor 上显现。Weight-only INT8 与 activations 同时量化也不是同一实验。

对照项	必须固定	可被改变
模型	同一 checkpoint/ONNX SHA、weights 与 preprocess	精度 / builder flags
输入	同一 raw input tensor / shape 集	benchmark order / repeated runs
环境	同 GPU/driver/TRT/CUDA/backend	若不同需另立实验，不合并统计
正确性	reference、atol/rtol、任务级阈值	candidate precision 与 calibration
性能	warmup、sync、sample count、percentile method	仅一个明确因素，如 FP32→FP16

F5. Dynamic Shape Profile 手算

以 TinyFFN `[B,T,8]` 为例，D=8 是固定特征维，B 是请求/批次行数，T 是序列长度。假设常见工作负载集中于 `[1,128,8]`，最大请求为 `[4,512,8]`，一个 profile 可以是 min `[1,1,8]`、opt `[1,128,8]`、max `[4,512,8]`。此声明意味着 build 允许某范围，但还要遵守模型维度间约束。`opt` 是优化器调优的代表点，不代表别的形状不能跑；min/max 外则不在合同范围内。

```
Input: x[B, T, 8]
Profile A for short prompts:
  min [1,   1, 8]  opt [1, 128, 8]  max [4, 256, 8]
Profile B for long prompts:
  min [1, 257, 8]  opt [1, 512, 8]  max [4, 2048, 8]

Test matrix:
  [1,1,8]          -> covered by A, expected pass
  [1,128,8]        -> A opt, expected pass
  [4,256,8]        -> A boundary, expected pass
  [1,257,8]        -> B boundary, expected pass
  [4,2048,8]       -> B max boundary, expected pass
  [5,128,8]        -> B dimension B exceeded, reject
  [1,2049,8]       -> T exceeded, reject
```

把请求按 T 分桶与 TensorRT Profile 是两层概念：Serving Batcher 决定同一批里哪些请求在 shape/layout 上可拼；Engine Profile 决定某个执行 Context 可以接受哪些 shape 范围并为哪些 shape 优化。Profile 覆盖范围宽不代表跨 T 的 request 就可以直接沿 B 拼接。若 batcher 对 T 不一致做 padding/mask，还需要模型语义和 mask 合同正确。

F6. Profile 越多越好吗？

多个 profile 可以更贴近常见 shape 分布，但会扩大 Engine build 管理复杂度、增加 Context/Profile 选择逻辑，可能增加构建时间或资源占用。先从真实流量统计 shape；如果绝大多数落在两个区间，两个 profile 可能足够；不要为每一个精确 T 建一个 profile。Engine 生成后用真实 shape histogram 检查 coverage，遇到 profile miss 时记录请求 shape、model version 和拒绝阶段。

深度辅导 G：TensorRT Runner 的异步时序与 C++ 所有权

G1. enqueue 是提交工作，不是同步拿结果

CPU 调用 enqueue 时通常把 kernel/拷贝排入 CUDA stream，函数能先返回；GPU 之后才依序执行。Host buffer 若被拷贝操作读取，至少活到异步 H2D copy 完成；device input 若被 kernel 读取，要活到 해당 stream 的执行完成；output buffer 若被后续 D2H 读取，要活到 output copy 完成。`enqueue returned=true` 说明工作成功排入，不等于输出值已能被 CPU 安全读取。

```
Host: fill pinned input -> cudaMemcpyAsync(H2D, stream)
      -> setTensorAddress(input, device_input)
      -> context.enqueueV3(stream)
      -> cudaMemcpyAsync(D2H, device_output, stream)
      -> record event(stream)
      -> wait/query event at client boundary
      -> read host_output; only now reuse/free buffers

Same-stream operations execute in stream order.
Cross-stream producer/consumer requires explicit event dependency.
```

G2. 资源所有权表

对象	创建者/所有者	释放条件	多线程风险
Engine	ModelVersionBundle	bundle retired 且 contexts/streams 不再引用	共享 Engine 通常可读；查版本契约
ExecutionContext	context pool slot	slot idle 且本次 enqueue 完成	不要并发改同一 context 状态
CUDA stream	worker/device runtime	stream 上任务完成并销毁	跨 stream 访问需 event 依赖
Input staging buffer	request/host buffer pool	H2D copy 已完成	异步传输期间不可覆写
Device input/output	device buffer pool	相关 Kernel/D2H 已完成	未完成时复用造成竞态
Completion event	request slot/stream owner	event 已同步/查询并消费	重用前应重新 record/清状态

G3. 同一 stream 的顺序图

```
stream 0: [H2D request A] -> [TensorRT A] -> [D2H A] -> [event A]
stream 1: [H2D request B] -> [TensorRT B] -> [D2H B] -> [event B]
          |                      |
          +-- independent if contexts/buffers are separate and GPU permits concurrency

Bad: stream 0 H2D(A), stream 1 TensorRT(A) without event/wait
     -> consumer may run before producer has finished writing input.
```

双 Stream 并不自动重叠。如果单个大 Kernel 占满 GPU，另一路只能排队；如果两个 stream 共用一份可变 input buffer，数据也会互相覆盖。要由 trace/Nsight/Event 证据确认重叠，而不是看到两个 stream 名称就宣称 overlap。

G4. C++ RAII 的资源释放顺序

RAII 把创建/销毁绑定到对象生命周期，适合 Engine/Context/Stream/Buffer；析构不应该在未知的异步 GPU work 尚未完成时释放资源。可设计一个 RequestExecution object 持有 VersionBundle、ContextLease、BufferLease、stream event。请求只有在 completion event 结束后才将这些 lease 归还池。异常路径同样要 wait/cancel/mark slot unusable，不能让异常越过后忘了释放。

```
class ExecutionLease {
 public:
  VersionHandle version;   // pins Engine while request is active
  ContextLease context;    // exclusive context slot
  BufferLease buffers;     // device/host memory lifetime
  StreamHandle stream;
  EventHandle done;

  Result wait_and_collect();
  // destructor must not recycle resources before done event completes
};
```

G5. 三种计时怎么各自定义

计时方式	起止点	包括什么	适合回答
Host end-to-end	submit→completed 或 runner enter→result ready	CPU schedule、queue、enqueue、wait、copy、mapping	客户端/服务端真实体验
CUDA Event	GPU stream 中 event start→event end	该 stream 两 event 间已执行的设备工作	GPU工作耗时/stream区间
Profiler trace	采集窗口内的 CPU/GPU op 与同步事件	探针与 trace overhead 可能存在	哪类活动/依赖出现在关键路径

CUDA Event 不包含它之前尚未进入 stream 的 CPU queue wait；Event 之间有其他 stream 并行时，单个 event interval 的含义也不是设备全局总工作量。Profiler 适合定位，不应直接当无探针 benchmark。每个报告要注明计时 API、同步策略、数据常驻状态和等待边界。

G6. Dynamic output 与内存分配

如果输出 shape 能由输入 shape 计算，可以先设置输入尺寸再推导输出尺寸，按需选择/分配 output buffer。若输出尺寸取决于数据内容（例如 nonzero 类操作），则输出大小未必能在执行前直接算出，需要 OutputAllocator 或 query/二阶段处理。预分配最大 shape 的内存更简单，但要衡量每个并发 slot 都留 max buffer 的 VRAM 代价。

G7. TensorRT Runner 验收合同

- Input contract：Tensor name、dtype、rank、shape range、layout、device、alignment/strides 要求。
- Ownership contract：每个 buffer/stream/context 谁拥有，异步完成由哪个 event 证明。
- Error contract：profile miss、shape infer fail、address missing、enqueue failure、CUDA async error 的映射。
- Result contract：output shape/dtype/finiteness/parity；一个 request 只完成一次。
- Shutdown contract：停止新 lease、等待 active events、回收 context/buffer、最后释放 Engine。
- Benchmark contract：build/startup/warmup/steady-state、host/E2E/event 三种 latency 独立报告。

> 最容易漏掉的是 asynchronous error：enqueue 调用点可能没有发现之后在 GPU stream 上发生的问题。需要在同步/查询 event 的边界检查 CUDA error，并将失败映射回对应 request_id 和 model_version。

深度辅导 H：已有 synthetic CSV 的逐步阅读与下一轮设计

这里不把数据曲线直接说成性能结论，而是示范怎样从 CSV 和源码开始审计。文件 `benchmarks/inferflow/10.11_capacity_samples.csv` 实际有 2600 条 request 样本、12 个 (T, max_batch_rows, concurrency) 配置，全部状态为 ok。每组样本数分别是 50、200、400，因为每 client 做 50 个请求且 concurrency 是 client 数。

H1. 先确认生成协议

- `std::barrier(concurrency + 1)` 让 clients 和主线程一起开始，减小启动时间差。
- 每个 client 顺序执行 submit→future.get→下一次 submit，所以 client 数固定，是 closed-loop。
- Runtime 是 DelayedRuntime，对每次 infer 调 `sleep_for(3ms)`，不是 CUDA，也不产生 GPU kernel。
- ServingEngine 的队列容量按 max(8, concurrency×4) 设置，batch 的 max wait 是 2ms。
- CSV 每 request 保存 queue/pre_bucket/batch/runtime/E2E；终端代码另外打印 QPS 与 runtime_calls，但 CSV 本身未保存这些 group-level 字段。

H2. 对照两个配置怎么读？

看 T=16、max rows=1，concurrency=1 的 E2E p50/p95/p99 为 3.851/5.588/8.807ms；concurrency=8 则为 33.004/46.344/58.124ms，Queue p95 从 0.311ms 到 41.349ms。此观察支持“在这次模型与协议中，串行 Runtime 接近饱和时队列 wait 迅速变长”，但它没有证明硬件 serving 的 capacity，也不支持推广为并发越大越差，因为 max rows/batching 在另一个配置中改变结果。

比较 T=16、max rows=4、concurrency=4，E2E p50/p95/p99 是 3.856/4.020/4.794ms；max rows=1 同 concurrency 的 E2E p95 为 16.485ms。可以提出 batching 假设，但必须补 runtime_calls 和实际 batch rows histogram，证明一次 infer 处理了几行。当前 CSV 的每条 request 都有一份 runtime_time，不能从相同 Runtime 调用被每个 batch request 映射后的时间戳简单数出实际调用次数。

H3. 为什么 QPS 不能只从这些行算？

QPS 需要 count / wall elapsed。CSV 有 group row count，却没有每组 run begin/end 或每条 submit/completion monotonic timestamp；`end_to_end_ms` 是单 request duration，不能推出整个 load window 的时长，因为这些请求并行重叠。源程序在运行时打印 elapsed 和 Runtime calls，但若终端日志没有归档，只有 CSV 就无法复现该输出。下一版 schema 应把每组 summary metadata 单独落 JSON，或在每行重复稳定 group fields：elapsed_ms、offered/accepted/completed/successful counts、runtime_calls、batch_size histogram、run_id、source revision。

H4. 统计数字是什么口径？

源程序 percentile 使用 nearest-rank：先排序，index=ceil(p*n)-1；并非 numpy 默认线性插值，也不是所有软件相同的 percentile method。若把 n=50 的 group p99 报到三位小数，它本质上通常就是最大值附近，数字的小数位不会增加统计确定性。10/11 个样本尤其不能支持稳健尾延迟结论。原始 CSV 应被视为首要证据，派生表必须记录 percentile method 和 filtering rule。

H5. 下一轮实验改动建议

缺口	新增字段/控制	为什么
吞吐无法复算	每组 monotonic elapsed 与 count summary JSON	QPS 是 count/time，时长不能从并发 E2E 相加
batch 效率未知	runtime_calls、每次 batch rows、partial/full flush reason	区分 max rows=4 真合批 vs 只配置上限
少轮次/噪声	至少 5 次交错 run，run_id 和 seed/order	减少机器状态/温度/先后顺序偏差
closed-loop 偏差	增加 open-loop fixed arrival rate + client schedule lag	观察持续过载和协调遗漏
measurement scope	明确 E2E clock、warmup、timer origin、deadline	防止不同运行使用不兼容口径
假硬件结论	证据标签 synthetic/cpu_runtime/cuda_runtime	Fake sleep 绝不能伪装 CUDA runtime

H6. 你应该怎样写 Observation？

正确的 Observation 只复述可核对的数据：例如“本 CSV 中 12 组共 2600 个状态 ok 样本；T=16、rows=1 时，concurrency=1 到 8 的 E2E p95 为 5.588ms 到 58.124ms，Queue p95 为 0.311ms 到 41.349ms；T=16、rows=4、concurrency=4 的 E2E p95 为 4.020ms。”它不能写“CUDA 吞吐提升 4 倍”，因为实验没有 CUDA，且缺少同组 QPS/runtime call 数据。

可证伪假设：max rows=4 可能因减少 Runtime 调用而降低延迟。下一轮固定 T/concurrency、交错比较 rows=1/4，并记录 runtime_calls 与 batch histogram；若证据不支持就修正假设。

按仓库落地的代码实验室：从接口到可验证结果

前面的章节讲了概念和验收合同；本章把'从哪里开始、先写哪一层、怎样验证'补齐。示例根据当前 Inferflow 结构设计：RequestContext/RequestTiming 在 cpp/inferflow/include/inferflow/request_context.h，服务入口与 worker 在 serving_engine.h/.cpp，同步执行与 batch 映射在 service_core.h/.cpp，FakeRuntime 测试在 cpp/inferflow/tests。这是教学方案，不表示这些新增模块已经存在。先保留 FakeRuntime 确定性路径，再接 metrics、tracing 或 TensorRT。

层	输入	产物	通过条件
1. 时间事实	RequestTiming 事件点	安全的 duration 纯函数	缺失/逆序/零时长定义明确
2. 指标聚合	每个 request 的终态 Result	低基数计数与延迟观察	终态只记一次，单位和标签固定
3. 请求上下文	request_id 与 trace carrier	跨 submit/queue/worker 的关联信息	同一请求 trace_id 不变
4. 控制面策略	queue/deadline/cancel/model version	拒绝、完成和生命周期状态	失败路径不悬挂、不越界
5. 真实 Runtime	稳定的 Runtime 契约	ORT 或 TensorRT adapter	正确性、环境和实测证据齐全

> 每层只引入一个主要变量。先在无网络、无 GPU 的单元测试中固定时间和调度，再接 exporter、SDK 或 TensorRT；否则一次失败很难区分是合同、线程时序还是环境/API 问题。

10.12 Lab：把 RequestTiming 变成可信 Metrics

现有 RequestTiming 使用 steady_clock::time_point，TimingMs 中 serialization_time 是 optional，因为当前 in-process engine 没有网络序列化边界。零初始化的 time_point 不能直接当'未发生'，否则漏写一个事件会生成错误 duration。做法是使用 optional timestamp，或明确的 event-present 位；不要依赖 time_point{} 表示 missing。

Step 1：先写纯 duration helper

```
using Clock = std::chrono::steady_clock;
using TimePoint = Clock::time_point;

std::optional<double> duration_ms(
    std::optional<TimePoint> begin,
    std::optional<TimePoint> end) {
    if (!begin || !end) return std::nullopt;
    if (*end < *begin) throw std::logic_error("timestamp order");
    return std::chrono::duration<double, std::milli>(*end - *begin).count();
}

// Derive each metric from its documented pair of events.
TimingMs derive_timing(const EventTimes& t) {
    TimingMs out;
    out.queue_time = require_ms(t.enqueued, t.dequeued);
    out.pre_bucket_wait = require_ms(t.dequeued, t.bucketed);
    out.batch_wait = require_ms(t.bucketed, t.dispatched);
    out.runtime_time = require_ms(t.runtime_begin, t.runtime_end);
    out.end_to_end = require_ms(t.submitted, t.completed);
    out.serialization_time = duration_ms(t.result_ready, t.serialized);
    return out;
}
```

产品代码里的 require_ms 应返回明确的错误或 optional，而不是 value_or(0)。若 Result 兼容性要求字段仍为 double，可在导出边界把缺失记录为 unavailable_count，不能把 0ms 当观测值。steady_clock 用来算进程内耗时；UTC wall time 用于日志时间戳和跨系统粗略排序。

Step 2：把计算与导出分开

不要让 ServiceCore 直接依赖 Prometheus SDK。先定义窄的 MetricsSink 接口：Counter increment、Gauge set、Histogram observe；用 fake sink 验证调用语义，再接线程安全 recorder。label 只允许枚举 backend/status/model family，不接受 request_id、trace_id、batch_id。

```
struct MetricLabels {
    std::string_view backend;     // fake, ort_cpu, ort_cuda, tensorrt
    std::string_view result_code; // ok, rejected, timeout, runtime_error
};

void record_terminal(MetricsSink& sink,
                     const MetricLabels& labels,
                     const Result& result) {
    sink.counter("requests_total", labels).increment();
    if (result.code == StatusCode::kOk)
        sink.counter("requests_succeeded_total", labels).increment();
    else
        sink.counter("requests_failed_total", labels).increment();
    sink.histogram("request_e2e_seconds", labels)
        .observe(result.timing.end_to_end / 1000.0);
    sink.histogram("runtime_seconds", labels)
        .observe(result.timing.runtime_time / 1000.0);
}
```

生产实现还应把 rejection、expiration、cancellation 分开统计；入口维护 offered/accepted；Queue depth/in-flight 是 Gauge，在 push/pop/dispatch/completion 对应事件更新；batch rows 用 Histogram。QPS 由 Counter 在固定 wall window 的增量计算，不是每请求 set 一个数字。

Step 3：单测怎样证明指标正确

测试输入	Arrange / Act	核心断言
完整成功请求	手工设事件为 0/1/4/6/10/16/21ms	Queue=3ms、pre-bucket=2ms、batch=4ms、runtime=6ms、E2E=21ms
入口拒绝	没有 enqueued/dequeued，直接终态 reject	queue 不记 0；rejected Counter 恰增 1
Runtime exception	FakeRuntime throw，写 runtime begin/end	runtime_error 增 1；终态只记录一次
逆序事件	end 比 start 早 1 tick	抛错/invalid；不出现负 latency
重复完成	同一 Pending 两次 finalize	第二次拒绝或幂等；Counter 不重复

建议新增纯逻辑 include/inferflow/timing_metrics.h 与 tests/test_timing_metrics.cpp，并在 Inferflow CMake 添加 inferflow_timing_metrics target。已有 test_service_core 覆盖 Runtime 错误映射；新测试不重复测线程，而是固定时间点检验算式。集成时在 ServingEngine 唯一 terminal/finalize 路径调用 sink，避免成功和错误分支重复打点。新增 target 后，在已配置的 WSL build dir 执行 `cmake --build cpp/build-cuda130 --target test_timing_metrics`，再用 `ctest --test-dir cpp/build-cuda130 --output-on-failure -R inferflow_timing_metrics`。

10.13 Lab：Trace Context 传播与精度 Parity

Trace 代码应该沿 RequestContext 走，而不是散落在 thread_local

ServingEngine 的 Pending 持有完整 RequestContext 并跨 queue 移动，所以最小传播点是在 RequestContext 增加轻量 trace carrier。SDK 对象可能有复杂生命周期，先保存标准化 trace_id/span_id/flags 或可复制的 SpanContext；不要把活动 Span 长时间放队列。每个阶段开始时创建自己的 span，结束时设置 status/attributes。Batch 关联多个 parent 时用 links 或显式 request id 列表。

```
struct TraceCarrier {
    std::string trace_id;
    std::string parent_span_id;
    std::uint8_t trace_flags{0};
};

struct RequestContext {
    std::string request_id;
    TensorInput input;
    TraceCarrier trace;
    // existing state, result and timing fields follow
};

auto span = tracer.start_span("queue.wait", ctx.trace);
span.set_attribute("request.id", ctx.request_id);
span.set_attribute("model.version", version_id); // bounded value
// Never attach prompt text, tensor values, or secrets.
```

传播顺序：入口提取/创建 context；submit 把它复制进 RequestContext；worker dequeue 时用 carrier 建立当前 span；RuntimeAdapter 记录 runtime span；完成路径设成功/错误并结束 span。thread-local 可在同步调用中暂存当前 span，但 queue 交接一定显式传 carrier。结束 SDK scope 后恢复旧 context，避免线程池复用时串 trace。

Trace 测试拆成四层

- 结构测试：submit 前 trace_id=trace-A，fake instrumentation 收到 queue/runtime/completed 事件，检查 trace_id 不变且 span_id 各自不同。
- 异步传播测试：一个线程 submit、另一个 worker 执行；只靠 thread-local 的实现应失败，显式 carrier 后通过。
- 错误路径测试：FakeRuntime 抛异常，runtime span 和 request log 有相同 trace_id，span status=error，future 仍 ready。
- Batch 测试：两个不同 trace 的请求组成一批，batch span 保留两个 links，输出仍按原 request 顺序映射。

精度 parity：锁输入、算误差、再测速度

代码上分成 correctness harness 与 performance harness。Correctness 固定 model SHA、样本、预处理、shape、reference 和容差；先验证 FP32 candidate，再只切一个 precision flag 跑 FP16。随机输入只做 smoke，代表性输入才支持任务质量判断。INT8 还要固定 calibration dataset SHA。性能 harness 在 warmup 后同步到同一完成边界，保存 raw latency、peak VRAM、build/startup 时间；async enqueue 的 CPU 返回耗时不是完成时间。

```
import numpy as np

def compare(reference, candidate, atol, rtol):
    ref = np.asarray(reference, dtype=np.float32)
    got = np.asarray(candidate, dtype=np.float32)
    if got.shape != ref.shape:
        raise ValueError(f"shape mismatch: {got.shape} != {ref.shape}")
    diff = np.abs(got - ref)
    denom = np.maximum(np.abs(ref), 1e-12)
    close = diff <= atol + rtol * np.abs(ref)
    return {
        "shape_equal": got.shape == ref.shape,
        "max_abs": float(diff.max()),
        "mean_abs": float(diff.mean()),
        "max_rel_eps_1e_12": float((diff / denom).max()),
        "nonfinite": int((~np.isfinite(got)).sum()),
        "failed_elements": int((~close).sum()),
    }
```

parity 测试需要覆盖哪些输入

类别	示例	目的
形状	最短/常见/最长 T，batch 1 和最大 batch	找 Profile、padding、动态维边界问题
数值	零、接近零、正负极值、正常分布	暴露下溢、溢出和近零相对误差问题
语义	真实代表性输入和固定预处理	观察模型任务指标而非只看 Tensor allclose
性能	固定 warmup、重复次数、同步点	比较同口径 steady-state 样本

测试必须同时检查 shape/dtype/finite，记录 atol/rtol、失败元素数和最大误差索引。只看 mean error 会掩盖少数灾难性错误；只看 relative error 会被 reference 接近零的元素放大。报告中把近零 epsilon 策略写清楚。

> 文件建议：在 request_context.h 增 carrier 字段；serving_engine 的 submit/worker 仅负责传播；新增独立 trace sink fake 测试。Parity 脚本放 TensorRT/ORT 实验目录，不要混进 Inferflow 核心 CMake 单测。

10.14 Lab：过载矩阵、Profile 边界与真实验证顺序

当前 tests/test_serving_engine.cpp 已覆盖队列满、重复 ID、排队到期、排队/运行中取消、Runtime 异常和 drain。先运行现有测试建立 baseline，再补遗漏项。已有 GateRuntime 用 condition_variable 控制运行时机，比固定 sleep 猜线程状态更稳定。

满队列测试的 Arrange / Act / Assert

```
// Deterministic sketch: active work is blocked inside FakeRuntime.
auto gate = std::make_shared<GateState>();
ServingEngine engine(std::make_unique<GateRuntime>(gate), 1);
auto active = engine.submit(make_request("active"), no_deadline);
CHECK(gate->wait_for_calls(1));
auto queued = engine.submit(make_request("queued"), no_deadline);
auto overflow = engine.submit(make_request("overflow"), no_deadline);
CHECK(await(overflow).code == StatusCode::kResourceExhausted);
CHECK(runtime_calls(gate) == 1); // overflow never reached Runtime
gate->open();
CHECK(await(active).code == StatusCode::kOk);
CHECK(await(queued).code == StatusCode::kOk);
engine.close();
```

容量细节：worker 正执行的 active 请求不计入 waiting queue 容量；capacity=1 时 active+queued 可以存在，第三个等待项才溢出。断言前先确认 active 已进入 Runtime，否则可能在 worker dequeue 前发生竞态。每个 future 使用有限 wait timeout；异常退出要用 scope guard 打开 Gate 并 close，避免测试自身挂死。还要验证 rejected request 的 ID 不留在 pending map。

基于当前 CMake target 的验证命令

```
# Run from the ai-infra-journey repository root on macOS.
cmake -S cpp -B cpp/build-mac -DBUILD_TESTING=ON \
  -DBUILD_INFERFLOW_BENCHMARKS=ON
cmake --build cpp/build-mac --target test_serving_engine \
  test_dynamic_batch inferflow_capacity_sweep -j
ctest --test-dir cpp/build-mac --output-on-failure \
  -R 'inferflow_(serving_engine|dynamic_batch|service_core)'
```

target 名来自当前 cpp/inferflow/CMakeLists.txt。configure 选项要以当前顶层 cpp/CMakeLists.txt 为准；若某 option 不存在则移除。先确认 configure 成功，再 build 指定 target；不要在未配置成功的目录运行 CTest，并把 ‘No tests found’ 当通过。Correctness 放 CTest，长时间容量扫描由单独 benchmark 执行。

Dynamic Shape：Profile 先作为纯合同测试

先在 Serving 输入校验层实现 covers(profile, shape) 纯函数，验证 rank 一致、每维 min≤actual≤max、固定维相等；该层不依赖 TensorRT。测试 min/opt/max 和越界 B/T。真实 backend smoke 再验证 Engine 构建、profile 选择、设置 concrete dims、推导 output dims、分配地址和 enqueue。两层分开才能定位范围逻辑错误与 TRT API/环境错误。

```
bool covers(const Shape& min, const Shape& max, const Shape& actual) {
    if (min.size() != max.size() || actual.size() != min.size()) return false;
    for (std::size_t d = 0; d < min.size(); ++d) {
        if (actual[d] < min[d] || actual[d] > max[d]) return false;
    }
    return true;
}

TEST(profile_boundaries) {
    EXPECT_TRUE(covers({1,1,8}, {4,512,8}, {1,1,8}));
    EXPECT_TRUE(covers({1,1,8}, {4,512,8}, {1,128,8}));
    EXPECT_TRUE(covers({1,1,8}, {4,512,8}, {4,512,8}));
    EXPECT_FALSE(covers({1,1,8}, {4,512,8}, {5,128,8}));
    EXPECT_FALSE(covers({1,1,8}, {4,512,8}, {1,513,8}));
}
```

covers 只证明边界逻辑，不能证明 TensorRT Engine 支持该 shape。真实 smoke 还要保存 TRT/GPU 版本、profile index、tensor name/dtype、shape inference、enqueue、同步后输出和 build log。API 版本可能改变 profile/地址接口，按安装版本文档实现。

10.15 Lab：热切换与 Runner 分成两个安全问题

先做版本注册表，再做模型加载

当前 IRuntime 是单一 unique_ptr，还没有 version bundle。先增加独立 ModelRegistry，不立即改 ServingEngine 所有权。用 mutex 保护 current shared_ptr；每个 RequestHandle 在 submit 时 acquire 一个 shared_ptr<const VersionBundle>，直到 future terminal 才释放。candidate 只有 load/validate/warmup/parity 全部通过后才能 publish。建议新建 include/inferflow/model_registry.h 和 tests/test_model_registry.cpp，并将 test_model_registry target 接到 BUILD_TESTING。

```
struct VersionBundle {
    std::string version_id;
    std::string model_sha;
    bool ready{false};
    std::shared_ptr<IRuntime> runtime;
};

class ModelRegistry {
public:
    std::shared_ptr<const VersionBundle> acquire() const {
        std::lock_guard<std::mutex> lock(mu_);
        return current_;
    }
    void publish(std::shared_ptr<const VersionBundle> next) {
        if (!next || !next->ready || !next->runtime) throw NotReady{};
        std::lock_guard<std::mutex> lock(mu_);
        current_ = std::move(next);
    }
private:
    mutable std::mutex mu_;
    std::shared_ptr<const VersionBundle> current_;
};
```

真实 bundle 还应包含输入/输出 schema、precision、profile 范围、context/buffer pool、backend/device 和 readiness。TensorRT 的 Engine 可由多个 context 执行；同一个 ExecutionContext 不能被多个线程并发使用。更容易漏掉的约束是：Engine 有动态 shape 时，并发使用的每个 context 必须使用独立的 Optimization Profile。因此并发槽数、profile 数与 profile 选择策略要一起设计；不能只建一个 profile 就创建多个并发 context。切换只是 current handle 替换；旧 bundle 由旧 RequestHandle 延长生命周期。

热切换测试按确定时间线写

```
registry.publish(ready_bundle("v1"));
auto old_request = registry.acquire(); // pins v1 before inference
registry.publish(ready_bundle("v2"));
auto new_request = registry.acquire(); // new work sees v2
CHECK(old_request->version_id == "v1");
CHECK(new_request->version_id == "v2");
CHECK(lifetime_probe(old_request) == LifetimeState::kAlive);
old_request.reset();                 // v1 can now be retired
```

场景	安排	通过条件
正常发布	v1 在途时发布 v2	旧请求输出来自 v1，新请求来自 v2
candidate 未 ready	v2 warmup/parity 失败再 publish	publish 拒绝，current 仍为 v1
回滚	v2 canary 后重新 publish v1	新请求回 v1；旧 v2 请求按策略结束
并发交换	多线程 acquire 同时切换版本	无半初始化 bundle；单请求只见一个版本
退役	持有弱引用并等待 lease 释放	最后一个旧 lease 释放后才析构资源

Runner 生命周期：先验证所有权状态机，再接 TensorRT

先用 FakeAsyncBackend 把所有权做对。每个执行 slot 拥有独占 ExecutionContext、CUDA stream、输入输出 device buffer 和 completion event；RequestExecution lease 持有 ModelVersion。提交后是 IN_FLIGHT，只有 completion event 被 query/synchronize 且异步错误检查结束，才能读结果、归还 buffer/context。异常路径要同步或把 slot 标记为不可复用。

```
validate input contract
acquire version lease + exclusive context/buffer slot
select profile; set input shape; infer output shape
enqueue H2D -> TensorRT execution -> optional D2H on owned stream
record completion event; keep request/version/context/buffers alive
wait or query event at result boundary; check asynchronous CUDA error
map output to request; fulfill promise exactly once
release context/buffer lease only after completion is proven
```

最小 Runner 测试不需要 GPU：FakeAsyncBackend 收到 submit 后阻塞在 latch；测试先尝试释放 buffer/context，断言 lease 仍持有资源；释放 latch、完成 event 后 future 返回，再允许资源归还。真实 TensorRT smoke 顺序是单输入、单 profile、FP32、单 stream、同步读取；通过后再测多 context。不要同时引入多 stream、FP16、动态输出和多个 profile。

每天收尾：四件证据必须齐

证据	写什么	不合格示例
代码差异	接口责任、实现位置、未解决问题	只写‘完成热更新’
测试输出	确切命令、pass/fail/skip、timeout	把 build 成功当 test 成功
数据	原始样本/Trace/Manifest/错误日志	只存截图或手抄一个 p95
结论	Observation / 可证伪假设 / 单变量下一实验	从 FakeRuntime 推断生产 GPU 容量

> 后端不可用时，仍完成纯逻辑、Fake backend、接口和 blocked 证据；但 Gate 必须标 blocked，不伪造 TensorRT 通过。

术语速查与资料来源

词	本讲中的意思
Cardinality	指标标签组合数；高基数会产生大量时间序列。
Histogram	按桶统计分布；分位数是从桶近似恢复，精度受 bucket 边界影响。
Admission control	入口决定是否接受新请求；与 dequeue/Runtime 调度不同。
Circuit breaker	失败过多时暂时停止调用后端，减少级联失败，并周期探测恢复。
Engine / Plan	TensorRT 编译/序列化后的网络执行产物。
ExecutionContext	运行时 shape、profile、地址等执行状态的载体。
Optimization Profile	动态输入允许的 min/opt/max shape 范围及优化参考形状。
Tactic	针对 layer 选择的具体实现/Kernel 策略；选择本身不是性能根因证明。
enqueue	将 GPU 工作排入某 CUDA stream；常为异步提交，不等于已完成。
Hot update	为新请求发布新版本，同时保持旧版本资源直到旧请求全部结束。

官方参考资料（离线讲义仍可阅读，在线 API 以安装版本为准）

1. NVIDIA TensorRT：Dynamic Shapes - Core Concepts
https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/dynamic-shapes-basics.html

2. NVIDIA TensorRT：Working with Dynamic Shapes
https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/work-with-dynamic-shapes.html

3. NVIDIA TensorRT Python API：IExecutionContext
https://docs.nvidia.com/deeplearning/tensorrt/latest/_static/python-api/infer/Core/ExecutionContext.html

4. NVIDIA TensorRT：How TensorRT Works / Threading
https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/how-trt-works.html

5. OpenTelemetry：Traces
https://opentelemetry.io/docs/concepts/signals/traces/

6. OpenTelemetry：Context Propagation
https://opentelemetry.io/docs/concepts/context-propagation/

7. Prometheus：Histograms and summaries
https://prometheus.io/docs/practices/histograms/

> TensorRT APIs and support evolve. This handout intentionally uses conceptual lifecycle plus pseudocode; before implementing, compare the exact C++/Python API with the TensorRT version installed on WSL. NVIDIA currently documents dynamic profile setup before input shape and enqueue, and profile ranges define permitted runtime shapes; APIs deprecated in older versions should not be copied blindly.

本讲义的工程状态说明：10.11 synthetic CSV 已检查并在本讲义中做了描述性分析；该数据仍是 FakeRuntime synthetic closed-loop，且 CSV 未持久化 elapsed/runtime_calls/batch histogram。10.12-10.15 的内容是依追赶计划组织的教学/执行指导，不是四天任务已完成的证明。